并行程序设计与算法（实验）

5-基于OpenMP的并行矩阵乘法

吴迪、刘学正

中山大学 计算机学院

实验概要

- **内容**
  - OpenMP程序编写、编译和运行
  - OpenMP并行区域与工作共享
  - 多线程并行矩阵乘法

- **目的**
  - 掌握OpenMP编程的基本流程
  - 掌握常见的OpenMP编译指令、运行库函数及环境变量
  - 分析调度方式对并行程序的影响

OpenMP 简介

- OpenMP（Open Multi-Processing）是一种用于**显式指导多线程、共享内存**并行计算的应用程序接口（API）。它由**编译器指令、运行时库函数**和**环境变量**三大部分组成，旨在简化并行编程，适用于多核 CPU 的并行计算任务

- **限制**
  - 并非所有厂商的实现完全相同
  - 不保证最高效利用共享内存
  - 不检查数据依赖、冲突、竞态条件或死锁
  - 不验证代码合规性
  - 不保证并行 I/O 的同步性

OpenMP 的设计初衷是提供简单易用的并行抽象，而非替代程序员思考，理解这些限制有助于编写正确、高效的并行代码

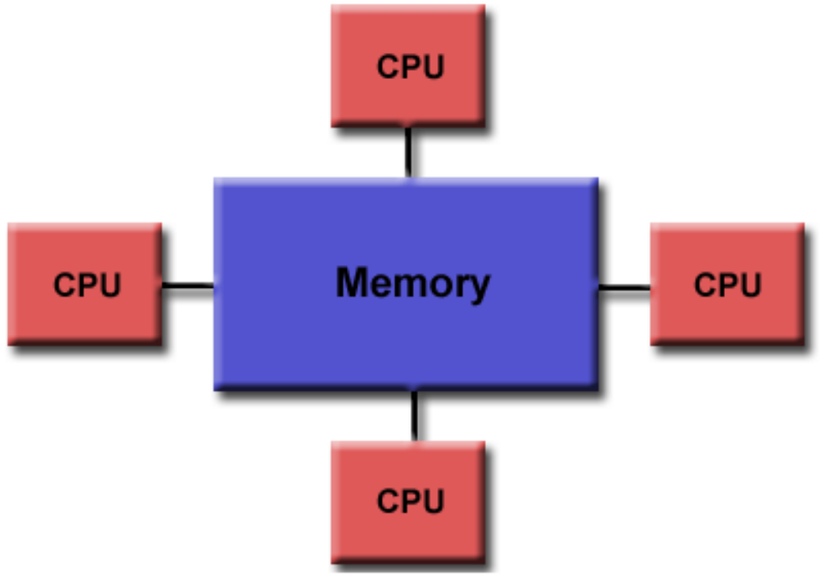
拓展学习资源：https://www.openmp.org/ （API规范、常见问题解答、演示、讨论等）

OpenMP 编程模型

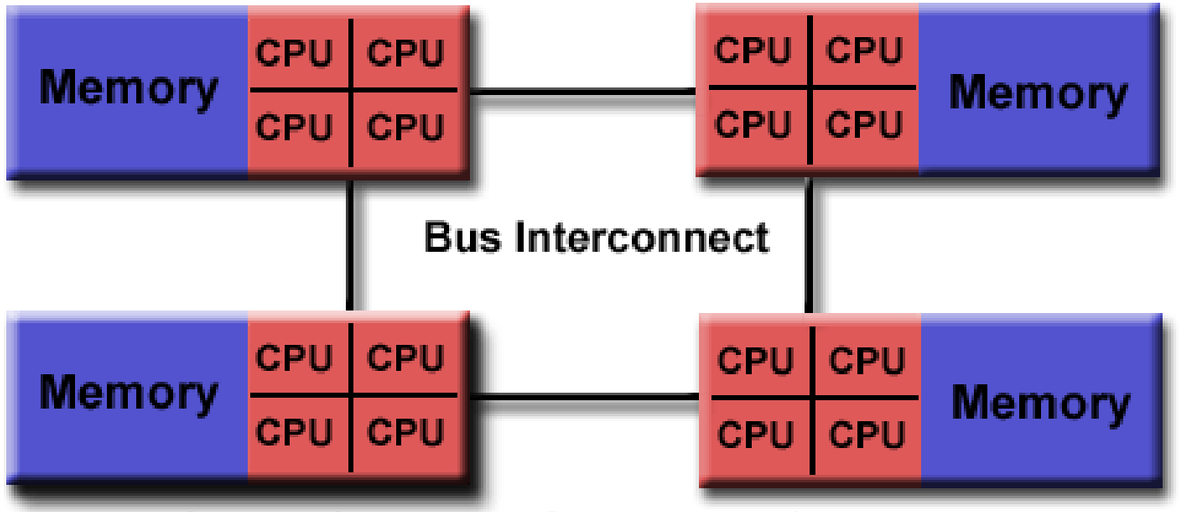


共享内存模型

- **在共享内存模型中，所有处理器核心共享一个统一的逻辑地址空间，并可直接访问同一物理内存池**
  - 底层架构可能采用UMA或NUMA，但是对程序员而言，内存表现为单一、共享资源



Uniform Memory Access(UMA)

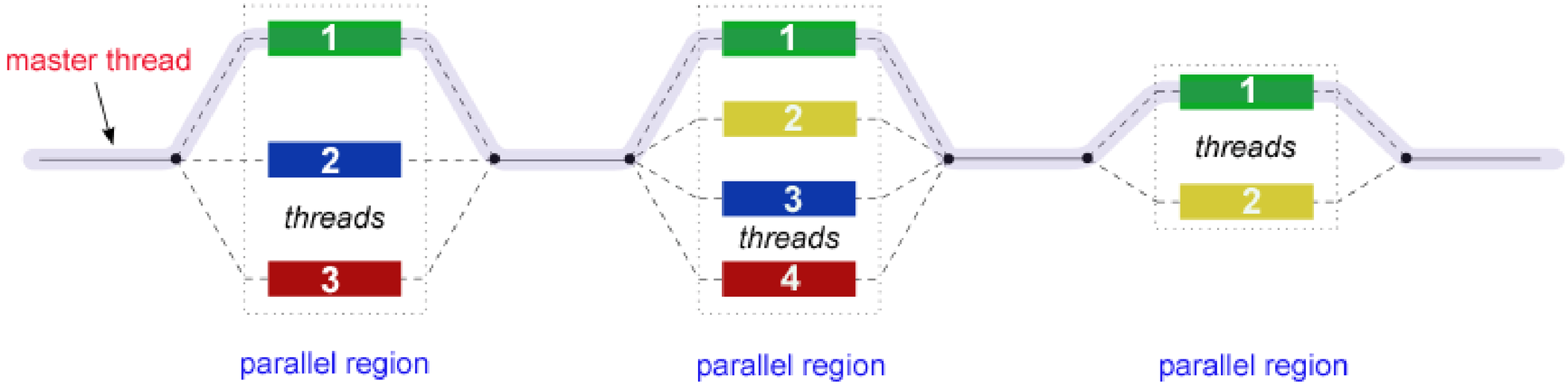


Non-Uniform Memory Access(NUMA)

图片来自：https://hpc-tutorials.llnl.gov/openmp/programming_model/

Fork-Join 模型

- OpenMP 采用 Fork-Join 模型：程序从单一线程开始；进入并行区域时，该线程与工作线程组成团队并发执行；退出并行区域后，仅主线程继续执行，其余线程同步并挂起



图片来自：https://hpc-tutorials.llnl.gov/openmp/programming_model/

OpenMP与Pthreads对比

核心差异对比表

维度	OpenMP	Pthreads
编程范式	声明式（基于编译指令）	命令式（基于函数调用）
执行模型	Fork-Join 模型	无固定模型
抽象层次	高级抽象	低级抽象
数据共享	隐式共享	显式共享
适用场景	规则数据并行、任务并行	通用并发编程：灵活但成本高

OpenMP程序基本结构

① 引入 OpenMP 头文件
- #include <omp.h>

② 并行区域创建：生成多个线程
- parallel

③ 数据作用域控制：
- private/shared/reduction/…

④ 工作共享构造：分配任务给线程
- for/sections/…

⑤ 调度机制
- static/dynamic/guided…

⑥ 同步机制
- barrier/critical/atomic/…

```
#include <omp.h>                                                    ①
#include <stdio.h>
#include <stdlib.h>

#define CHUNKSIZE   10
#define N       100

int main(int argc, char *argv[]) {
    int nthreads, tid, i, chunk;
    float a[N], b[N], c[N];

/* Some initializations */
    for (i = 0; i < N; i++)
        a[i] = b[i] = i * 1.0;
    chunk = CHUNKSIZE;
                    ②                                   ③
    #pragma omp parallel shared(a, b, c, nthreads, chunk) private(i, tid)
    {
        tid = omp_get_thread_num();
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }
        printf("Thread %d starting...\n", tid);
                    ④           ⑤
    #pragma omp for schedule(dynamic, chunk)
        for (i = 0; i < N; i++) {
            c[i] = a[i] + b[i];
            printf("Thread %d: c[%d]= %f\n", tid, i, c[i]);
        }

    }⑥/* end of parallel section */
}
```

示例程序：omp_workshare1.c

OpenMP程序的编译与运行

命令行编译与运行

```
# 编译命令
gcc -o omp_hello omp_hello.c  -fopenmp
# 运行命令
OMP_NUM_THREADS=4 ./omp_hello
```

运行结果

```
coder@4da13e446a5f:~/project$ OMP_NUM_THREADS=4 ./omp_hello
Hello World from thread = 0
Number of threads = 4
Hello World from thread = 1
Hello World from thread = 2
Hello World from thread = 3
```

示例程序：omp_hello.c



OpenMP 的核心组件与工具

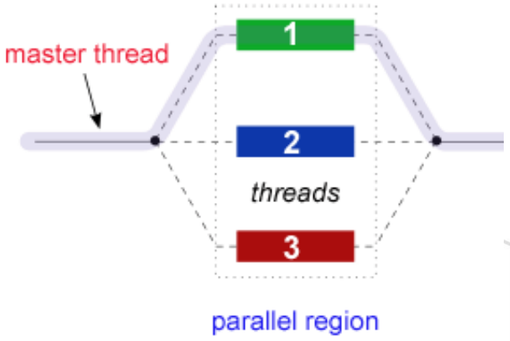
- **编译器指令**
  - 格式：#pragma omp directive-name [clause[[,] clause]...]
  - 功能：管理并行区域、分配并行任务、同步线程等
- **运行时库函数**
  - 设置/查询线程数：omp_set_num_threads(int) / omp_get_num_threads()
  - 查询线程ID：omp_get_thread_num()
  - 同步与互斥：omp_set_lock() / omp_unset_lock()
  - 获取运行时间：omp_get_wtime()
- **环境变量**
  - 设置线程数： export OMP_NUM_THREADS=8
  - 决定线程调度策略: export OMP_SCHEDULE="dynamic"

官方文档：https://www.openmp.org/wp-content/uploads/OpenMP-4.5-1115-CPP-web.pdf

并行区域构造指令

并行区域构造：创建线程团队

- parallel 是 OpenMP 的核心指令，后跟一个结构化代码块，指示编译器将**代码块并行化**
- 主线程**创建一组线程**，组内每个线程都完整执行该结构化代码块，块结束后所有线程同步，仅主线程继续执行后续代码



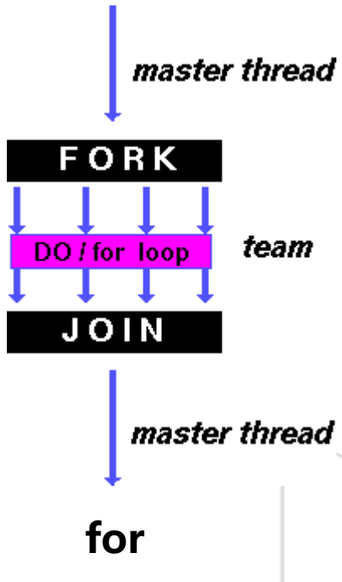

```
#pragma omp parallel [clause[ [, ]clause] ...]
    structured-block

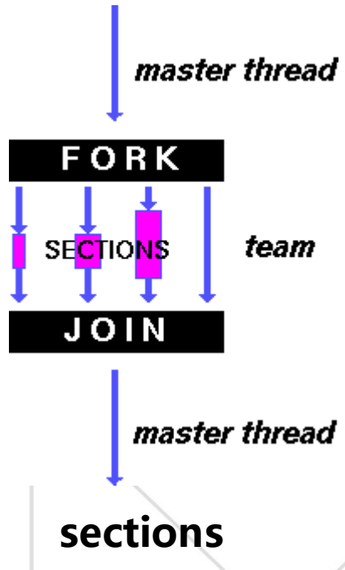
clause:
        if([ parallel : ] scalar-expression)
        num_threads(integer-expression)
        default(shared | none)
        private(list)
        firstprivate(list)
        shared(list)
        copyin(list)
        reduction(reduction-identifier: list)
        proc_bind(master | close | spread)
```

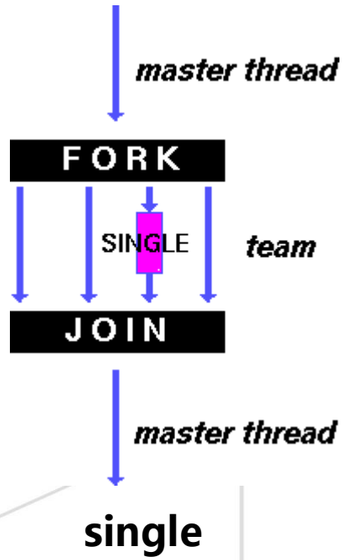
子句 **proc_bind** 用于控制线程与处理器绑定策略：
- **master**: 所有线程都绑定在主线程所在的位置
- **close**: 线程以紧凑的方式分配，优先靠近主线程
- **spread**: 线程尽量分开，均匀分布到不同区域

工作共享构造指令

- **工作共享构造：在已有线程团队中分配任务**
  - #pragma omp for —— 将循环迭代轮流分配给各线程
  - #pragma omp sections —— 将不同代码块（section）分配给不同线程
  - #pragma omp single —— 指定一个线程执行，其他线程跳过







循环并行

#pragma omp for

#pragma omp for *[clause[[,]clause] ...]*
for-loops

clause:

private(*list*)
firstprivate(*list*)
lastprivate(*list*)
linear(*list [: linear-step]*)
reduction(*reduction-identifier* : *list*)
schedule(*[modifier [, modifier] :]* *kind*[, *chunk_size*])
collapse(*n*)
ordered[(*n*)]
nowait

示例程序：omp_workshare1.c

```
/*** Do matrix multiply sharing iterations on outer loop ***/
/*** Display who does which iterations for demonstration purposes ***/
printf("Thread %d starting matrix multiply...\n", tid);
#pragma omp for schedule (static, chunk)
for (i = 0; i < NRA; i++) {
    printf("Thread=%d did row=%d\n", tid, i);
    for (j = 0; j < NCB; j++)
        for (k = 0; k < NCA; k++)
            c[i][j] += a[i][k] * b[k][j];
}
```

自动将 for 循环的迭代任务均匀划分给并行区内的多个线程执行

任务段并行

#pragma omp sections

```
#pragma omp sections [clause[ [, ] clause] ...]
    {
    [#pragma omp section]
        structured-block
    [#pragma omp section
        structured-block]
    ...
    }

clause:
    private(list)
    firstprivate(list)
    lastprivate(list)
    reduction(reduction-identifier: list)
    nowait
```

```
#pragma omp parallel shared(a, b, c, d, nthreads) private(i, tid
{
    tid = omp_get_thread_num();
    if (tid == 0) {
        nthreads = omp_get_num_threads();
        printf("Number of threads = %d\n", nthreads);
    }
    printf("Thread %d starting...\n", tid);

    #pragma omp sections nowait
    {
        #pragma omp section
        {
            printf("Thread %d doing section 1\n", tid);
            for (i = 0; i < N; i++) {
                c[i] = a[i] + b[i];
                printf("Thread %d: c[%d]= %f\n", tid, i, c[i]);
            }
        }

        #pragma omp section
        {
            printf("Thread %d doing section 2\n", tid);
            for (i = 0; i < N; i++) {
                d[i] = a[i] * b[i];
                printf("Thread %d: d[%d]= %f\n", tid, i, d[i]);
            }
        }

    }  /* end of sections */

    printf("Thread %d done.\n", tid);

}  /* end of parallel section */
```

代码划分成多个独立的 section 代码段，由并行区域的线程自动分配、并发执行不同代码段，实现任务级并行

示例程序：omp_workshare2.c

数据作用域属性子句

OpenMP数据作用域属性子句对照表

子句	作用	适用场景
private	为每个线程创建变量的私有副本，初始值未定义	需要线程独立操作变量，避免竞争
shared	变量在所有线程间共享，所有线程访问同一内存地址	多线程安全地共享只读数据，或协作写入互不冲突的内存位置
default	设置变量的默认作用域：例如 default(shared) 或 default(none)	统一管理未显式声明作用域的变量
firstprivate	类似 private，但继承外部变量的初始值	需要线程私有化变量且保留外部初始值
lastprivate	类似 private，但将最后一次迭代（或最后一个 section）的值回写主线程	需要保留并行循环或 sections 的最终结果
copyin	将主线程的 threadprivate 变量值广播到所有线程的私有副本	统一初始化所有线程的全局线程私有变量
copyprivate	将 single 区域内的变量值广播到其他线程的私有副本	需要将主线程的局部计算结果同步到其他线程
reduction	对变量执行归约操作（如求和、最大值），自动合并所有线程的结果	需要高效合并线程局部结果（如累加、极值、逻辑运算）

数据作用域属性子句

基础场景：共享与私有变量

```
#pragma omp parallel shared(a, b, c, nthreads, chunk) private(i, tid)
    {
        tid = omp_get_thread_num();
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }
        printf("Thread %d starting...\n", tid);

#pragma omp for schedule(dynamic, chunk)
        for (i = 0; i < N; i++) {
            c[i] = a[i] + b[i];
            printf("Thread %d: c[%d]= %f\n", tid, i, c[i]);
        }

    }  /* end of parallel section */
```

这些变量在所有线程间**共享**，所有线程访问同一份内存地址，适合**只读或受控修改**的场景

每个线程拥有变量的**独立私有副本**，线程间互不干扰，从根源上避免数据竞争

示例程序：omp_workshare2.c

进阶场景：规约操作

```
sum = 0.0;

#pragma omp parallel for reduction(+:sum)
  for (i=0; i < n; i++)
    sum = sum + (a[i] * b[i]);
```

每个线程维护 sum 的私有副本，并行结束时**自动合并**所有线程的结果，无需手动加锁 / 同步

示例程序：omp_reduction.c

调度与任务分配子句

OpenMP 调度与任务分配子句对照表

子句	作用	适用场景
schedule(static[, chunk])	在编译时将循环迭代静态均分给线程（可指定块大小 chunk）	循环各次迭代计算量均衡，追求最低调度开销
schedule(dynamic[, chunk])	在运行时动态分配迭代块，线程空闲时领取新任务	循环各次迭代计算量不均衡（如稀疏矩阵、递归深度不同）
schedule(guided[, chunk])	动态分配，但初始块大、逐渐变小，减少调度次数	迭代工作量未知或变化剧烈，需平衡负载与调度开销
schedule(auto)	由编译器/运行时系统自动选择最优调度策略	开发者不愿手动调优，信任运行时优化（性能可能不稳定）
schedule(runtime)	调度策略由环境变量 OMP_SCHEDULE 在程序启动时决定	需在不重编译的情况下测试不同调度策略
collapse(n)	将嵌套的 n 层循环扁平化为单层循环再并行	嵌套循环总迭代数少，单层并行度不足
ordered	保证循环中带 #pragma omp ordered 的代码段按原顺序执行	需并行计算但部分结果必须顺序输出（如打印日志）

调度与任务分配子句

调度示例：动态调度

– 迭代任务被分成多个块，线程空闲时主动请求新的任务块

```
// Parallelise the for loop using the dynamic schedule
#pragma omp parallel for schedule(dynamic)
for(int i = 0; i < 10; i++)
{
    printf("Thread %d processes iteration %d.\n", omp_get_thread_num(), i);
}

printf("With a chunksize of 2:\n");

// Parallelise the for loop using the dynamic schedule and chunks of 2 iterations
#pragma omp parallel for schedule(dynamic, 2)
for(int i = 0; i < 10; i++)
{
    printf("Thread %d processes iteration %d.\n", omp_get_thread_num(), i);
}
```

示例程序：omp_sched_dynamic.c

默认块大小为 1，每次分配 1 个任务，负载均衡性最好，但调度开销稍大

每次分配 N（例如2）个任务，减少调度次数，可平衡负载均衡和调度开销

调度方式	任务分配示例
schedule(dynamic)	线程 0: 0,2,3,4,5,6,7,8,9；线程1：1
schedule(dynamic,2)	线程 0: 0-1,4-5,6-7,8-9；线程1：2-3

同步构造指令

常用同步指令详解

#pragma omp critical *[(name)* **[hint (***hint-expression***)]]**
structured-block

创建一个“临界区”，确保同一时间**只有一个线程**能执行这段代码

#pragma omp barrier

强制所有线程**在此处等待**，直到所有线程都执行到这一行

#pragma omp atomic *[***seq_cst***[,]] atomic-clause [[,]***seq_cst***]*
expression-stmt

#pragma omp atomic *[***seq_cst***]*
expression-stmt

对**单个变量**的操作实现原子操作

#pragma omp flush *[(list)]*

强制写回内存，解决**缓存一致性**问题

更多指令 https://www.openmp.org/wp-content/uploads/OpenMP-4.5-1115-CPP-web.pdf
使用指引 https://rookiehpc.org/openmp/docs/index.html

运行库函数

常用运行库函数详解

功能分类	函数原型（C）	功能描述
线程管理	int omp_get_num_threads(void);	返回当前并行区域中的线程数
	void omp_set_num_threads(int);	设置后续并行区域的默认线程数
	int omp_get_thread_num(void);	返回当前线程的ID（0到num_threads-1）
	int omp_get_max_threads(void);	返回并行区域可能使用的最大线程数
锁同步	void omp_init_lock(omp_lock_t *);	初始化一个锁
	void omp_set_lock(omp_lock_t *);	获取锁（阻塞）
	void omp_unset_lock(omp_lock_t *);	释放锁
	void omp_destroy_lock(omp_lock_t *);	销毁锁
时间函数	double omp_get_wtime(void);	返回当前时间（单位：秒）
	double omp_get_wtick(void);	返回时钟精度（单位：秒）
任务调度	void omp_set_schedule(omp_sched_t, int);	设置循环调度的策略（如static/dynamic）和分块大小
	void omp_get_schedule(omp_sched_t *, int *);	获取当前调度策略

环境变量

常用环境变量详解

功能分类	环境变量名	功能描述	合法取值
线程与并行控制	OMP_NUM_THREADS	设置并行区域的默认线程数	非负整数
	OMP_DYNAMIC	允许运行时动态调整并行区域线程数	true, false
	OMP_NESTED	启用或禁用嵌套并行	true, false
调度与循环控制	OMP_SCHEDULE	设置循环默认调度策略及块大小	static[,chunk] dynamic[,chunk] guided[,chunk] auto
线程绑定与亲和性	OMP_PROC_BIND	设置线程与CPU核心的绑定策略	true, false, master, close, spread
	OMP_PLACES	定义线程可绑定的硬件位置	threads、cores、sockets

更多环境变量请参考： https://www.openmp.org/wp-content/uploads/OpenMP-4.5-1115-CPP-web.pdf

实验内容

OpenMP通用矩阵乘法

- 使用OpenMP实现并行通用矩阵乘法
- 设置线程数量（1-16）、矩阵规模（128-2048）、调度方式
  - 调度方式包括默认调度、静态调度、动态调度
- 根据运行时间，分析程序并行性能
- 选做：根据运行时间，对比使用OpenMP实现并行矩阵乘法与使用Pthreads实现并行矩阵乘法的性能差异，并讨论分析。

Questions?

